05_performance_by_isolation_level

Propietario	© Carlos Roberto
Verificación	Verificada
Etiquetas	Arquitectura de Datos

Impact by Isolation Level Measures logical reads and execution time for the same query under different isolation levels.

5.1 Stores measurements for comparison

```
-- ============================================================
-- C1300S03 | Script 05 — Performance Impact by Isolation Level
-- Measures logical reads and execution time for the same query
-- under different isolation levels.
-- SQL Server 2019+ | Database: STN_Lab
-- ============================================================

USE STN_Lab;
GO

-- ============================================================
-- RESULTS TABLE — stores measurements for comparison
-- ============================================================
IF OBJECT_ID('tempdb..#PerfResults', 'U') IS NOT NULL DROP TABLE #P
erfResults;
GO

CREATE TABLE #PerfResults (
    IsolationLevel  VARCHAR(50),
    ElapsedMS       INT,
    LogicalReads    INT,
    Notes           VARCHAR(200)
);
GO
```

```sql
-- ============================================================
-- BENCHMARK QUERY — same query for all tests
-- Simulates a typical report: total payments per period
-- ============================================================

-- TEST 1 — READ UNCOMMITTED
PRINT '--- READ UNCOMMITTED ---';
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
GO

SELECT
    d.PeriodoFiscal,
    COUNT(p.PagoID)         AS TotalPagos,
    SUM(p.MontoPagado)      AS TotalMonto
FROM dbo.Declaracion d
JOIN dbo.Pago p ON p.DeclaracionID = d.DeclaracionID
WHERE d.PeriodoFiscal BETWEEN '2023-01' AND '2023-12'
GROUP BY d.PeriodoFiscal
ORDER BY d.PeriodoFiscal;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
-- Read the logical reads and elapsed time from Messages tab, recor
d below:
INSERT INTO #PerfResults VALUES
('READ UNCOMMITTED', 0, 0, 'No blocking. Risk: dirty reads. Fastest
but unreliable.');
GO

-- TEST 2 — READ COMMITTED (default, no RCSI)
PRINT '--- READ COMMITTED ---';
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
GO
```

```sql
SELECT
    d.PeriodoFiscal,
    COUNT(p.PagoID)         AS TotalPagos,
    SUM(p.MontoPagado)      AS TotalMonto
FROM dbo.Declaracion d
JOIN dbo.Pago p ON p.DeclaracionID = d.DeclaracionID
WHERE d.PeriodoFiscal BETWEEN '2023-01' AND '2023-12'
GROUP BY d.PeriodoFiscal
ORDER BY d.PeriodoFiscal;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
INSERT INTO #PerfResults VALUES
('READ COMMITTED', 0, 0, 'Default. Blocks on conflicting writes. Sa
fe for most reads.');
GO

-- TEST 3 — REPEATABLE READ
PRINT '--- REPEATABLE READ ---';
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
GO

SELECT
    d.PeriodoFiscal,
    COUNT(p.PagoID)         AS TotalPagos,
    SUM(p.MontoPagado)      AS TotalMonto
FROM dbo.Declaracion d
JOIN dbo.Pago p ON p.DeclaracionID = d.DeclaracionID
WHERE d.PeriodoFiscal BETWEEN '2023-01' AND '2023-12'
GROUP BY d.PeriodoFiscal
ORDER BY d.PeriodoFiscal;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
INSERT INTO #PerfResults VALUES
```

```sql
('REPEATABLE READ', 0, 0, 'Holds shared locks until TX end. Prevent
s non-repeatable reads. High contention risk.');
GO

-- TEST 4 — SNAPSHOT ISOLATION
PRINT '--- SNAPSHOT ISOLATION ---';

-- Enable SI first (only needs to be done once)
ALTER DATABASE STN_Lab SET ALLOW_SNAPSHOT_ISOLATION ON;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SET TRANSACTION ISOLATION LEVEL SNAPSHOT;

BEGIN TRANSACTION;

SELECT
    d.PeriodoFiscal,
    COUNT(p.PagoID)         AS TotalPagos,
    SUM(p.MontoPagado)      AS TotalMonto
FROM dbo.Declaracion d
JOIN dbo.Pago p ON p.DeclaracionID = d.DeclaracionID
WHERE d.PeriodoFiscal BETWEEN '2023-01' AND '2023-12'
GROUP BY d.PeriodoFiscal
ORDER BY d.PeriodoFiscal;

COMMIT;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
INSERT INTO #PerfResults VALUES
('SNAPSHOT', 0, 0, 'Row versioning. Non-blocking reads. TempDB pres
sure. Detects update conflicts.');
GO

-- TEST 5 — SERIALIZABLE
PRINT '--- SERIALIZABLE ---';
```

```sql
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;

SELECT
    d.PeriodoFiscal,
    COUNT(p.PagoID)         AS TotalPagos,
    SUM(p.MontoPagado)      AS TotalMonto
FROM dbo.Declaracion d
JOIN dbo.Pago p ON p.DeclaracionID = d.DeclaracionID
WHERE d.PeriodoFiscal BETWEEN '2023-01' AND '2023-12'
GROUP BY d.PeriodoFiscal
ORDER BY d.PeriodoFiscal;

COMMIT;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
INSERT INTO #PerfResults VALUES
('SERIALIZABLE', 0, 0, 'Range locks held. Prevents phantoms. Maximu
m blocking risk. Use sparingly.');
GO

-- ============================================================
-- DISPLAY RESULTS TEMPLATE
-- Update ElapsedMS and LogicalReads with values from Messages tab
-- ============================================================
SELECT
    IsolationLevel,
    ElapsedMS       AS [Elapsed (ms)],
    LogicalReads    AS [Logical Reads],
    Notes
FROM #PerfResults
ORDER BY CASE IsolationLevel
    WHEN 'READ UNCOMMITTED' THEN 1
    WHEN 'READ COMMITTED'   THEN 2
```

```
    WHEN 'SNAPSHOT'         THEN 3
    WHEN 'REPEATABLE READ'  THEN 4
    WHEN 'SERIALIZABLE'     THEN 5
END;
GO

/*
====================================================
EXPECTED RESULTS PATTERN (to fill after execution)
====================================================

Isolation Level       | Elapsed (ms) | Logical Reads | Notes
----------------------|--------------|---------------|-------------
--------------------------------
READ UNCOMMITTED      | ~lowest      | same          | Skips shared
locks — logical reads identical
READ COMMITTED        | baseline     | same          | Acquires/rel
eases shared locks per row
SNAPSHOT              | ~similar     | +overhead     | Reads from v
ersion store in TempDB
REPEATABLE READ       | +higher      | same          | Holds locks
longer — contention increases
SERIALIZABLE          | ~highest     | same          | Range locks
— blocks insertions during read

KEY INSIGHT:
Logical reads are nearly identical across levels for the same quer
y.
The real cost difference shows up under CONCURRENCY (multiple sessi
ons),
not in single-session benchmarks.
For true performance impact, run Script 02 and Script 04 under loa
d.
====================================================
*/

-- ============================================================
-- RESET to safe default
-- ============================================================
```

```sql
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
GO
```

Isolation Level	Elapsed (ms)	CPU (ms)	Logical Reads	Physical Reads
READ UNCOMMITTED	304	358	23,509	0
READ COMMITTED	88	608	23,887	0
REPEATABLE READ	137	438	23,887	0
SNAPSHOT	137	687	23,887	123
SERIALIZABLE	232	688	23,887	112